Technologies for IoT Ecosystem – A.A. 2025/26

Hardware Laboratory

Part 1

Introduction to the Arduino environment and first simple sketches

---

1.1) LED Driving

Write a sketch that drives two LEDs, so they blink with independent periods. The sketch must meet the following specifications:

- The pins to which the two LEDs are connected, and the blinking (semi-)periods, must be defined using constants.
- To drive the first LED, use the `delay()` function within the `loop()`.
- To drive the second LED, use an Interrupt Service Routine connected to the MCU Timer1 (using the `MBED_RPI_PICO_TimerInterrupt` library or the `Scheduler` library).

As a test, use blinking periods that are not multiples of each other (e.g. 3s and 7s) and connect the two LEDs to pins D2 and D3 on the board.

1.2) Communication via Serial Port

Modify the previous sketch to transmit the current LED states via serial (upon request). The sketch must meet the following specifications:

- At startup, a Serial communication with the PC must be configured with BaudRate = 9600 bit/s, and a "welcome" message must be sent to the PC once the connection is established.
- In each iteration of the `loop()` function, the sketch must check for the reception of a character via serial.
    - If the received character corresponds to the ASCII code of 'R' (red LED) or 'L' (green LED), the sketch must send a message to the PC containing the state of the corresponding LED.
    - If the received character does not match one of the two previous ones, an error message must be printed.

Verify correct operation using the Arduino IDE Serial Monitor.

During debugging, keep in mind that serial printing occurs synchronously with the switching of one of the two LEDs (the one managed inside `loop()`), therefore the value will always be sent immediately before or immediately after a switching event. Also make sure to use the `volatile` qualifier for variables used both in the ISR and in the `loop()` function.

Extra: Try using the `Scheduler` library introduced in the previous exercise to make the status printing asynchronous with respect to the main loop.

Figure 1: Example of serial output for Exercise 1.2

1.3) Presence Detection via PIR Sensor

Write a sketch capable of counting the number of people passing in front of the Arduino using a Passive InfraRed (PIR) motion sensor. The sketch must meet the following specifications:

- The digital output of the PIR sensor must be connected to a digital input of the Arduino. The sketch must react to value changes on the pin using an ISR active on both rising and falling edges.
- The ISR must read the sensor value and reproduce it on an LED.
  That is, the LED must be ON when the PIR sensor outputs a HIGH value, and OFF otherwise.
- Additionally, the ISR must store in a variable the total number of events detected by the PIR sensor.
  An event is defined as a new movement detected by the sensor.
- The `loop()` function must send every 30 seconds a message to the PC containing the total number of recorded events.

Verify correct operation using the Arduino IDE Serial Monitor.

Figure 2: Example of serial output for Exercise 1.3

1.4) DC Motor (Fan) Control via PWM

Write a sketch capable of controlling the rotation speed of a DC motor using Pulse-Width Modulation (PWM). The sketch must meet the following specifications:

- The fan control pin must be connected to an Arduino output that supports PWM.
- In `setup()`, the sketch must set the rotation speed to 0 by writing the value 0 (duty cycle = 0%) to the control pin.
- The `loop()` function must process serial input:
  - Increase speed when receiving the character '+'
  - Decrease speed when receiving the character '-'
  - Any other character must generate an error message
- The sketch must define a fixed number of speed "steps" (for example, 10), each corresponding to an increasing duty cycle, up to a maximum of 100%, corresponding to value 255 in the PWM register.
  Each '+' or '-' increments or decrements the speed by one step.
- When maximum or minimum duty cycle is reached, further '+' or '-' inputs must generate a message indicating that the maximum/minimum speed has already been reached.

During testing, note that at low duty-cycle values (depending on the number of steps chosen), the motor may not generate enough torque to start rotating if initially stopped (although it may maintain rotation if already moving). This behavior is normal and must not be considered a program error.

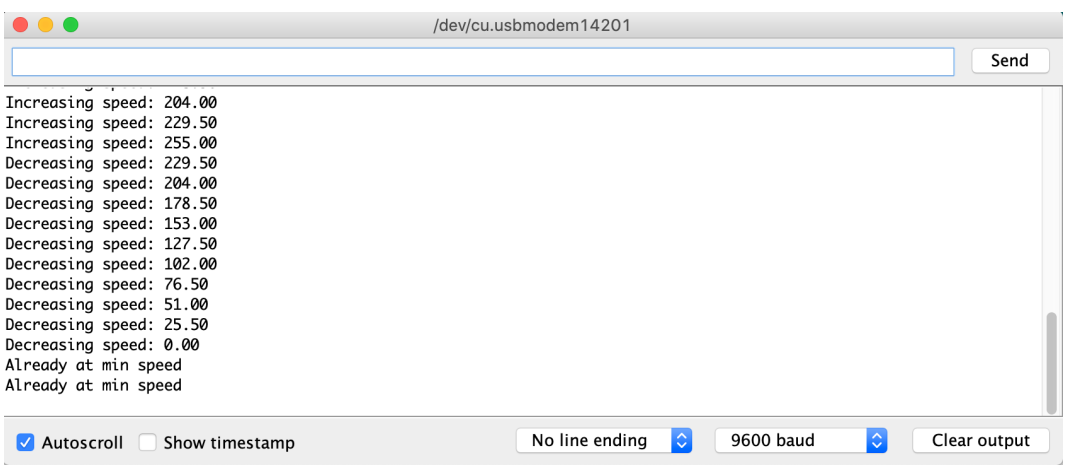

Figure 3: Example of serial output for Exercise 1.4

1.5) Reading Analog Values (Temperature)

Write a sketch capable of periodically monitoring ambient temperature and sending the value to the PC via serial port. The sketch must meet the following specifications:

- The temperature sensor signal pin must be connected to one of the Arduino analogue inputs (for example, A0).
- The voltage value of the pin must be read periodically (for example, every 10 seconds) and converted into temperature using the appropriate functions provided in the sensor datasheet.
- The measured value must be sent to the PC via the serial port.

To test the sketch, the sensor may be artificially heated (without damaging it!), for example, by gently blowing on it.

Figure 4: Example of serial output for Exercise 1.5

1.6) Communication with a Smart Actuator (LCD Display) via I²C

Modify the previous sketch so that temperature information is displayed on an LCD rather than transmitted to the PC. Use the `LiquidCrystal_PCF8574` library (available in the Arduino IDE Library Manager) to interface with the display. The sketch must follow the same specifications as the previous one, except that the information must be displayed on the LCD. Explore the source code of the provided library to identify the most efficient way to update the display temperature, minimising the amount of data sent over the I²C bus.

Figure 5: Example of LCD output for Exercise 1.6